

Reinforcement Learning-Based Balancing and Control Across Different OWSBR Platforms

Kwan Ip (Edward) Chan, Osvaldo Aldaz, Kai Xiang (Arthur) Chong

Date: June 05, 2026

Abstract

This paper focuses on the autonomous navigation and balancing of a one-wheeled self-balancing robot (OWSBR). Initial work starts by treating the OWSBR as a two-wheeled self-balancing robot (TWSBR), acting like an inverted pendulum on a cart. PPO, SAC, and DDPG algorithms were used to train the OWSBR to balance and traverse towards a reference point, with linear motion constrained to 1D (forwards and backwards). The algorithms converged, and the OWSBR was able to reach its goal and remain upright. A reaction wheel was then added to allow the OWSBR to yaw, consequently allowing the OWSBR to traverse a 2D plane (Model 2). A PPO algorithm was attempted to train this new model, but learning from scratch was deemed infeasible and/or inefficient.

Introduction

Self-balancing robotic platforms, including two-wheel self-balancing robots (TWSBRs), ballbots, and one-wheel self-balancing robots (OWSBRs), have been studied for balancing and mobility applications [1]–[6]. The one-wheel self-balancing robot tackles the same challenge by keeping an upright body balanced on a wheel. Like the TWSBR, the OWSBR behaves as an inverted pendulum on a wheel, where the forward and backward torque keep the body from falling. This has the benefit of having a reduced footprint, at the cost of adding a reaction wheel to control the yawing in the horizontal plane and improves balancing [5], [6]. Traditionally, controllers for such systems have relied on PID, LQR, backstepping control, robust control, adaptive nonlinear control, and fractional-order PID control techniques [1], [2], [4], [7]–[9]. These methods perform well near the equilibrium point, but performance may degrade under large disturbances, nonlinear motion, model uncertainty, or different operating conditions.

Reinforcement learning (RL), on the other hand, provides a data-driven alternative for problems where developing an accurate model is hard. RL algorithms can learn policies through interacting with a simulated environment [10]–[14]. Proximal Policy Optimization (PPO) is widely used because of its stable on-policy learning and clipped policy update structure [13]. Soft Actor-Critic (SAC) is an off-policy maximum-entropy reinforcement learning algorithm that encourages exploration while improving sample efficiency [15]. Deep Deterministic Policy Gradient (DDPG) is an off-policy actor-critic algorithm designed for continuous action spaces [16]. Despite the novelty of the papers, these works typically evaluate a single RL algorithm and do not compare with multiple algorithms under identical conditions. Hence, it is unclear how the choice of algorithms affects learning speed and control quality.

This project aims to bridge the gap by developing and comparing three deep RL algorithms, PPO, SAC, and DDPG, on two different newly developed OWSBR models within the same MuJoCo simulation. All

three controllers are trained on identical tasks, driving the robot from a start position to a goal while maintaining upright balance. By keeping everything else constant, the comparison shows how these algorithms affect the sample efficiency, training time, and success rate. Each trained policy is verified across different random seeds to assess the generalization of the algorithms, and the results of these models are compared in terms of learning curves and convergence speed.

Methods

Model Development

Model 1



Fig. 1. Model 1 design in Mujoco.

This is a simple OWSBR model that only consists of a ground wheel and a chassis mounted on the wheel. The mass of the chassis is 4.75 kg, and the wheel is 0.32 kg, making the total mass to be 5.07 kg. The wheel is 0.075m in radius and is on the ground with a friction coefficient of 2.0 to provide traction between the wheel and the ground. It implements the Euler integrator, which is a simple and fast integrator. At each time step of 0.002s, it uses the current state and steps forward to the next state with its current derivative. It is designed to only move forward and backward in one-dimensional space. We assigned an objective in which the robot must travel from a starting position of $x = 0$ m to a goal position of $x = 2$ m. A training episode terminates itself if the robot falls ($|\theta| > 0.5$ rad) or travels beyond the position limit ($x > 2.8$ m). The objective is considered achieved when the model reaches the goal within 0.1 m with a tilt angle less than 0.25 rad.

Model 2

The second version of the OWSBR (Model 2) is similar to Model 1, adding a reaction wheel with a radius of 0.1 m, a height of 0.05 m, and a mass of 0.5kg. The control torque of the reaction wheel is limited to ± 5 Nm. The reaction wheel can be seen on the top of the model seen in Figure 2 below.

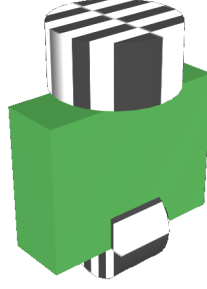


Fig. 2. Model 1 Design. The added reaction wheel is on top, with the original chassis and ground wheel.

Simulation Environment

All training and evaluations are carried out in the MuJoCo environment. The OWSBRs are modeled as single-wheel inverted pendulums with a wheel for forward and backward motion control. At the start of each episode, the robot is placed upright at the start position with near zero-velocity and a small random angular deviation to allow policy from only memorizing a single initial condition.

Observation and Action Space

The agent observes 5 different quantities at each timestep, such as the horizontal position, horizontal velocity, pitch angle, and pitch angular velocity. The action is a single continuous torque bounded between $[-5, 5]$ Nm, passing through a Tanh function.

Reward Function

The reward function comprises a progress term proportional to the reduction in distance to the set goal. A set of penalties and rewards is included in the reward function. For example, a small distance penalty is included to discourage stalling, and a pitch exceeding 0.20 rad will activate a tilt penalty. Besides that, the episode will terminate with -20 if the robot falls ($|\theta| > 0.5$ rad or $|x| > 2.8$ m), and a reward of +50 points are awarded upon reaching the goal within 0.10m with a tilt below 0.25 rad.

Proximal Policy Optimization (PPO)

PPO is an on-policy RL algorithm. It initially collects a rollout of experience, then estimates the advantages of each action using Generalized Advantage Estimation (GAE), then updates the policy by a clipped objective, which prevents the policy from differing too much in an update. An entropy bonus is used to encourage exploration. The entropy coefficient decays linearly from 0.01 to 0.002 and is included to encourage the model to shift from exploration to exploitation. It helps the policy to explore more initially and stabilize later.

For our case, it collects a rollout of 2048 timesteps, computes advantages, and updates the policy for 10 epochs after each rollout. The network is a shared two-layer system branching into actor and critic. The actor outputs the mean of a Gaussian action distribution, while the standard deviation is stored as a single learned parameter clamped between $[0.1, 2.0]$ for model 1 and $[e^{-1.5}, e^{0.7}]$ for model 2. These clamps control the amount of exploration. The sampled action is then passed through a tanh function and scaled to the torque range.

Soft Actor-Critic (SAC)

SAC is an off-policy RL algorithm, where every transition is stored in a replay buffer and is reused for learning, making it more sample-efficient than PPO. SAC uses a stochastic actor. Instead of outputting a single fixed action, the policy outputs a probability distribution. The distribution sampled an action, then passed it through a tanh function and scaled it to a torque. The twin critics (Q-networks) are learned simultaneously to evaluate the future action values and return scalar Q values. The minimum of the two is used for critics to mitigate overestimation bias, while also providing stable value estimation [17]. A target critic is maintained via soft updates ($\tau = 0.005$).

For our case, the entropy coefficient is tuned to a target entropy of 0 and -1 for Models 1 and 2, respectively. Actions, on the other hand, are sampled uniformly at random for the first 5000 steps before training begins.

Deep Deterministic Policy Gradient (DDPG)

DDPG is an off-policy algorithm that learns a deterministic policy. Unlike SAC, DDPG is paired with a single Q-network. The actor outputs a single torque value through a tanh squashing function, with no standard deviation or sampling involved. Exploration is added by adding noise to the action during training.

For both of the models, Gaussian noise is added to the action during training, decaying linearly from 0.3 to 0.1 over the first 1, 000, 000 steps. Both the actor and critic are updated via soft updates ($\tau = 0.005$).

Results and Discussion

RL Training on Model 1

Model 1 was trained on a single seed and verified against at least three seeds using a Lenovo Yoga 7i (CPU only). All three algorithms were also given the same initial condition and goal position, providing a baseline for comparing convergence speed, training stability, and policy quality.

Figure 3 shows that SAC learned the task fastest in terms of environment steps. Its reward became positive at around 250,000 steps and reached a peak reward of 29.0. PPO reached the highest peak reward of 30.9, but its training curve was less stable because it dropped significantly early before recovering to

reach a positive reward later. DDPG had the weakest training performance. Its peak reward was 18.5 and only stayed positive for small amounts of steps, else the reward stayed mostly negative during training. The DDPG critic loss also increased sharply near the end in the training loss plot, which suggests that its value estimation became unstable.

The wall-time plot shows that PPO trained the fastest at 54.8 minutes, followed by SAC at 84.7 minutes and DDPG at 190.7 minutes. This suggests that SAC was the most sample-efficient, PPO was the most time-efficient, and DDPG was the least reliable during training because it required the longest training time but did not achieve a positive peak reward.

Additional training and verification results are available in the link to code section under the `owsbr/Ed_` results files.

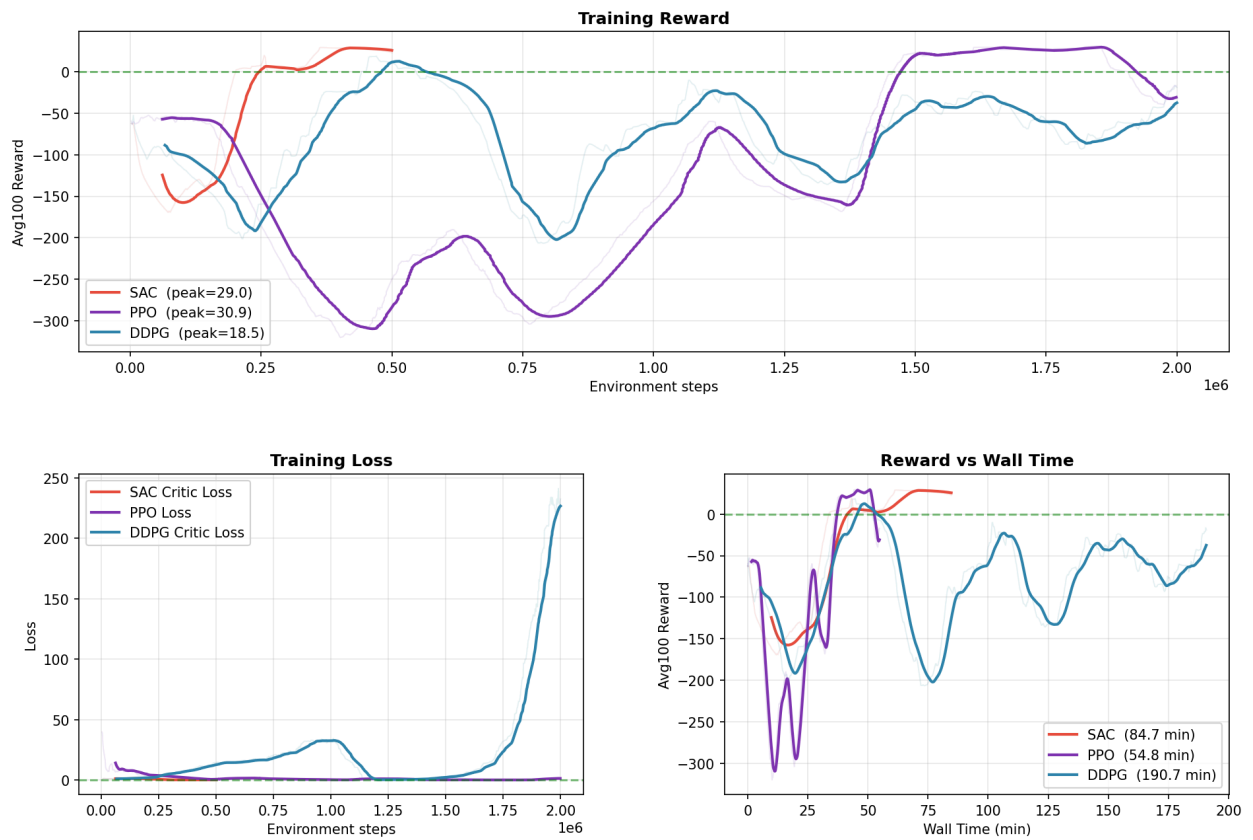


Fig. 3. Training reward (top), training loss (bottom-left), and reward vs. wall-clock time (bottom-right) for SAC (red), PPO (purple), and DDPG (blue) on Model 1

Figure 4 shows that all three algorithms were able to move the robot toward the goal region while keeping the tilt angle within the stable range. PPO reached the goal region slightly faster but used the highest torque RMS of 1.34 Nm, which suggests that it used more aggressive control to reach the goal faster. SAC had a smooth trajectory with a lower torque RMS of 1.09 Nm. DDPG had the lowest torque RMS of 1.07 Nm, but it showed large torque and angular acceleration spikes near the end of the episode. This suggests that DDPG is less stable when decelerating.

All three policies kept the tilt angle within the stable range of plus or minus 20 degrees for most of the episode. This means that each algorithm learned some form of balancing behavior. However, the angular velocity and angular acceleration plots show that SAC had the smoothest overall motion, while DDPG had the most noticeable terminal oscillations. PPO was stable overall, but it used a little bit more control effort than SAC.

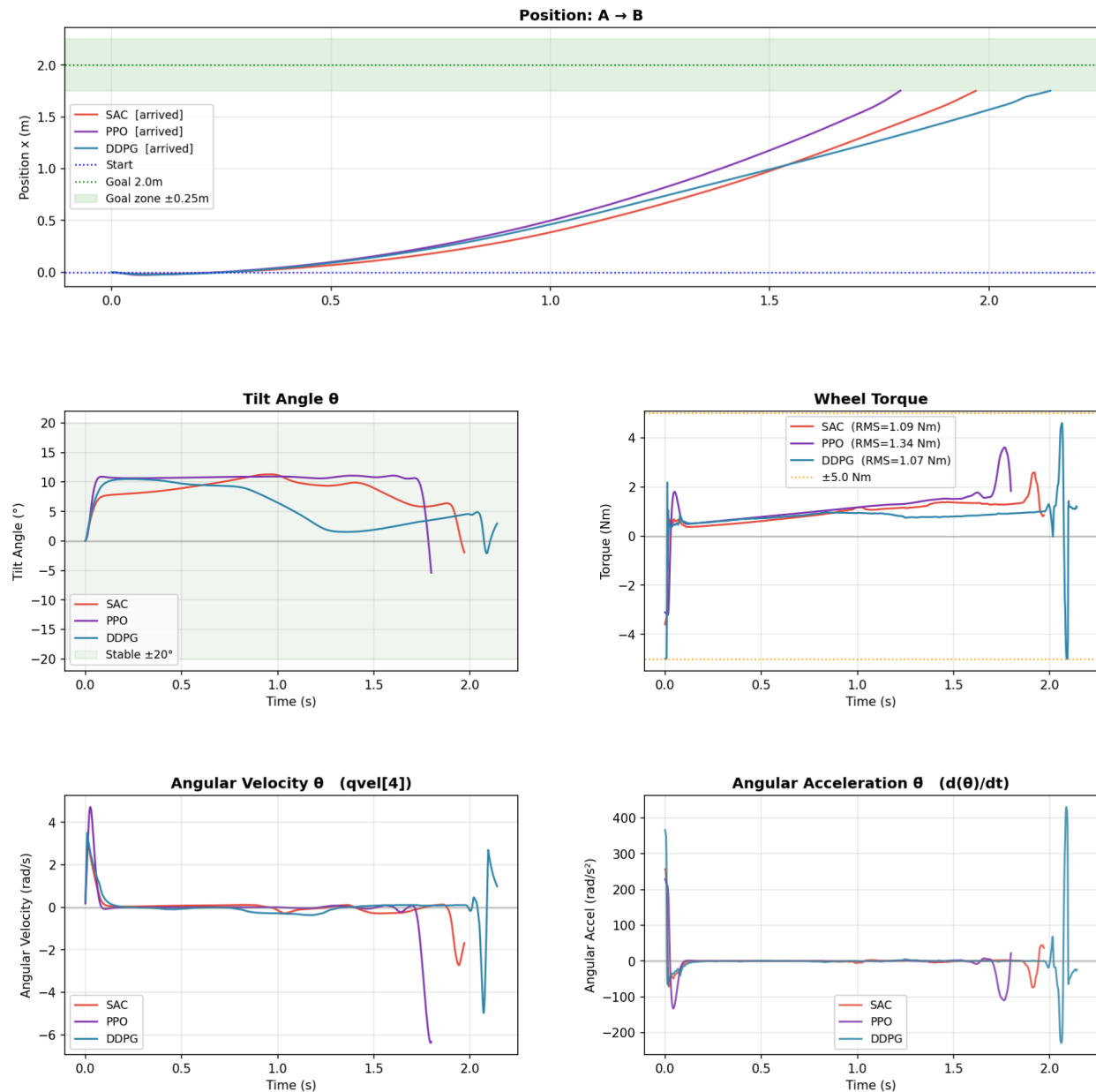


Fig. 4. Deterministic episode traces for SAC, PPO, and DDPG on Model 1, showing position (top), tilt angle and wheel torque (middle), and angular velocity and angular acceleration (bottom).

RL Training on Model 2 (Reaction Wheel Disabled)

Model 2 was trained on a single seed and verified against at least three seeds using a MacBook Air M3 (CPU only). All three algorithms (PPO, SAC, and DDPG) were also given the same initial condition and goal position, providing a baseline for comparing convergence speed, training stability, and policy quality.

All three algorithms were initially assigned 500, 000 environment steps, but PPO and DDPG failed to converge within the budget and were extended to 2, 000, 000 steps. Figure 5 compares the training dynamics. Based on the training reward plot, SAC converges in the fewest steps, crossing positive reward by approximately 250, 000 steps, and reaching a peak of 24.9. PPO reaches the highest peak reward (28.6) at around 1, 000, 000 steps and plateaus, but it completes training in only 6.8 minutes, roughly five times faster than SAC (34.3 minutes) and six times faster than DDPG (44.8 minutes). DDPG shows high instability, where the reward function plot oscillates between -250 and +26 throughout the full 2, 000, 000 steps without settling. These results illustrate a trade-off between the three different methods, where SAC is the most sample-efficient, PPO is the most compute-efficient, and DDPG is the least reliable among the three. Additional training and verification videos are available in the GitHub repository under [Arthur/Ozzy_Segway_Data/Videos](#).

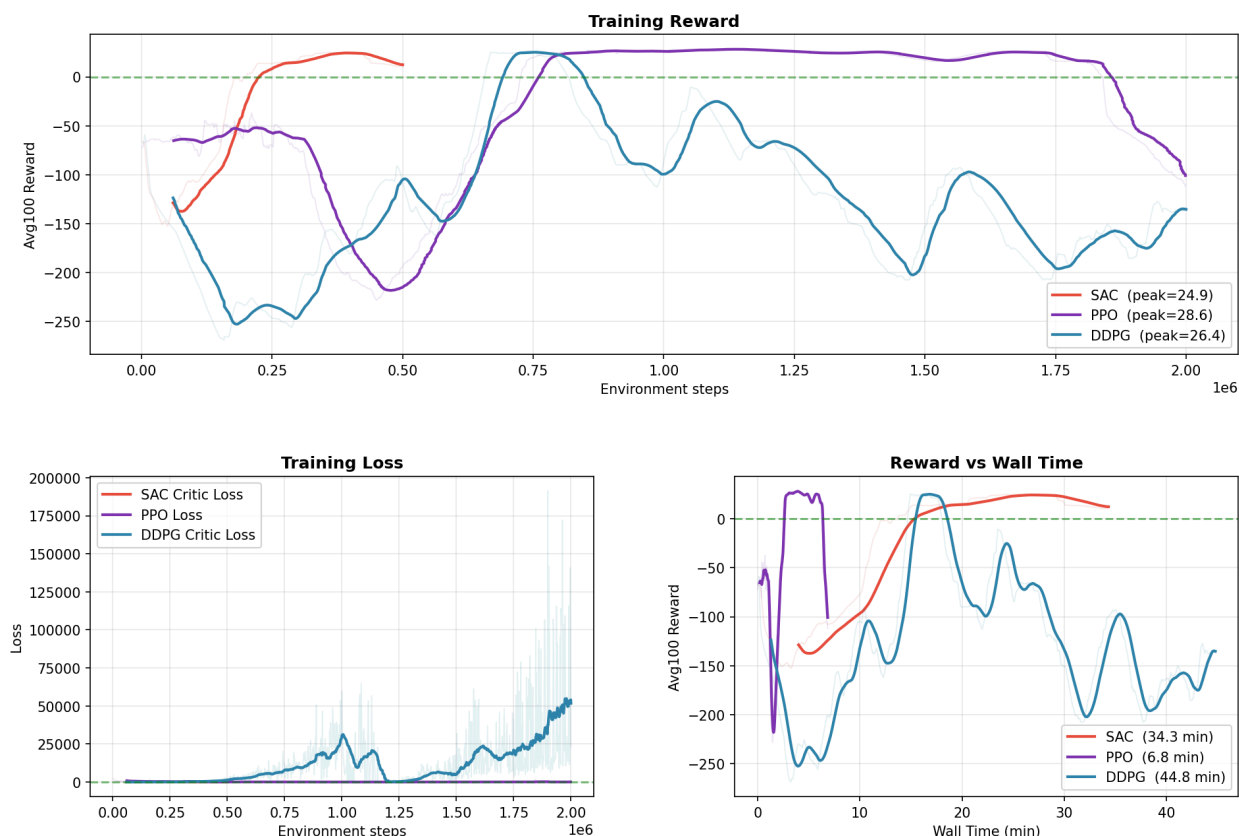


Fig. 5. Training reward (top), training loss (bottom-left), and reward vs. wall-clock time (bottom-right) for SAC (red), PPO (purple), and DDPG (blue) on Model 2.

Despite these differences in training stability, all three algorithms produced policies that successfully navigated to the goal, as shown in Figure 6. DDPG reaches the goal zone (within 0.25 m of the 2.0 m target) fastest at around 1.8 seconds, followed by PPO at 2.0 seconds and SAC at 2.2 seconds. The quality of the control, on the other hand, is different. SAC archives the smoothest trajectory with the lowest torque RMS (1.10 Nm), while DDPG requires significantly more aggressive commands (1.77 Nm). Besides that, DDPG also shows high-frequency oscillations in both torque and angular acceleration signals. PPO falls between the two at 1.20 Nm. All three policies maintain a tilt within the $[-20, 20]$ degree stable region, but DDPG's angular velocity shows large instability near arrival, showing that it is less confident during the transition from forward motion to stopping at the goal.

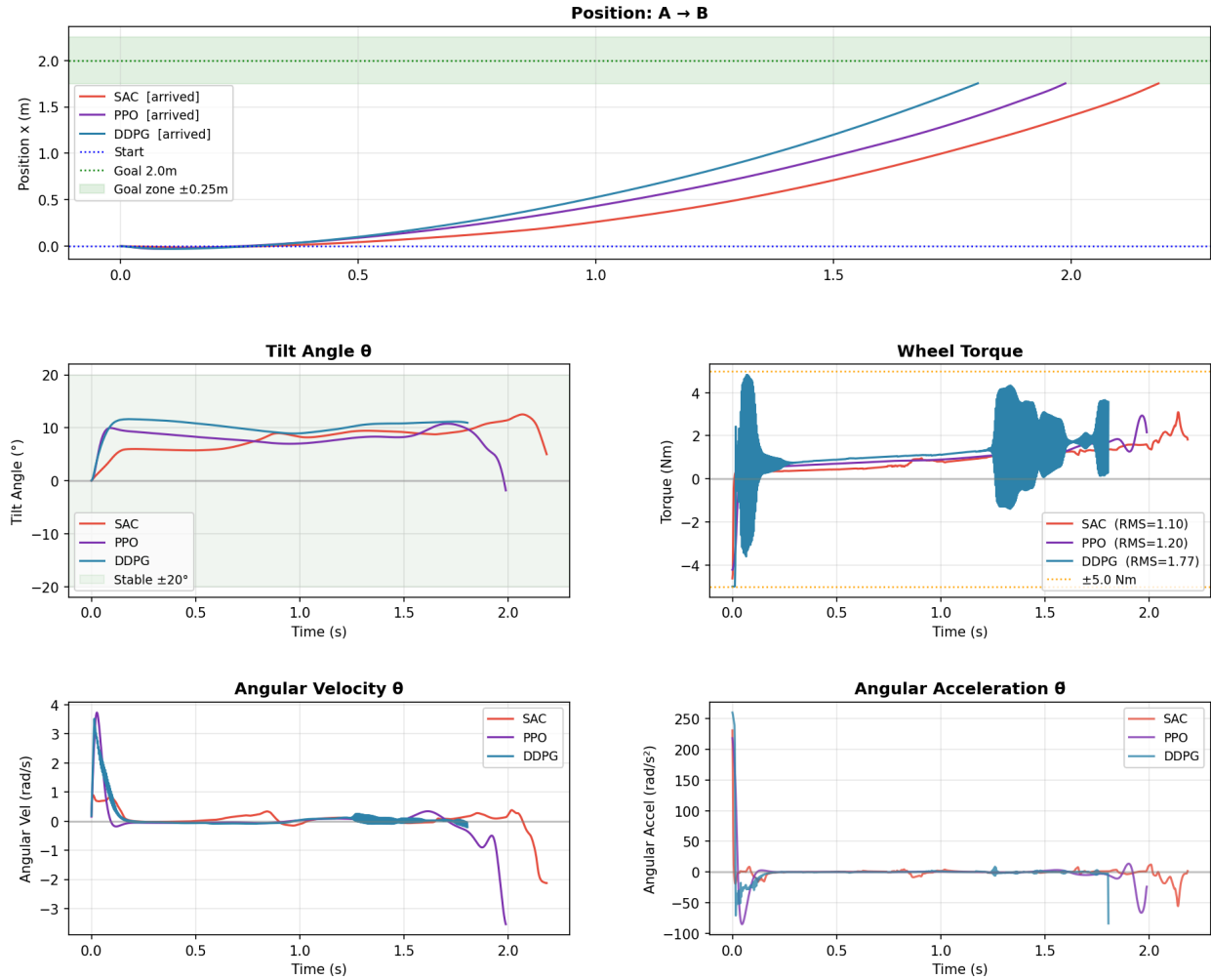


Fig. 6. Deterministic episode traces for SAC, PPO, and DDPG on Model 2, showing position (top), tilt angle and wheel torque (middle), and angular velocity and angular acceleration (bottom).

Overall, both model 1 and model 2 show that PPO and SAC performed better than DDPG. SAC was the most sample-efficient because it reached positive reward with fewer environment steps and produced the smoothest final control. PPO was the most time-efficient because it trained much faster and reached the

highest peak reward. DDPG was able to reach the goal, but its reward curve, critic loss, torque signal, and angular acceleration were less stable. Therefore, SAC is better for smooth and reliable control, while PPO is better when shorter training time is more important.

RL Training on Model 2 (Reaction Wheel Enabled)

A PPO algorithm was run, but it converged in an undesirable state. Figure 7 compares the training dynamics. The reward plateaus around -45.9 points, and the rest of the training is stuck at this local maximum. Assuming that adding complexity to the PPO algorithm would help capture more complexity with traversing a 2D environment, more resources were allocated, with 30 million environment steps, increased actor/critic network sizes, and number of layers, with negligible improvement. Tuning hyperparameters and the reward function also proved ineffective.

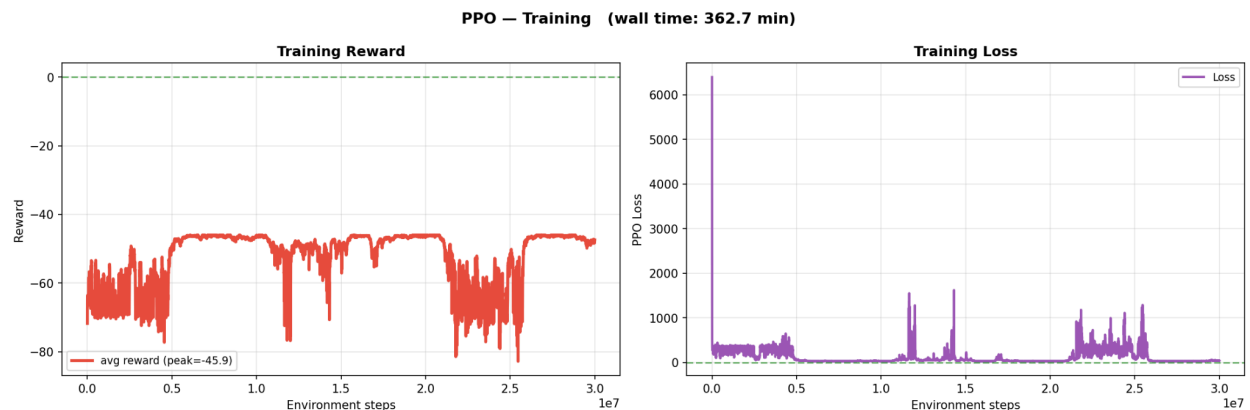


Figure 7. Reward and Loss during training. This was the most complex training, yet performed just as well (or as badly) as smaller trials, with 30 million steps and an expanded actor/critic net.

In the simulation, the OWSBR learns to fall towards the goal, quickly gaining some reward, but struggles to learn any other strategy. Figure 8 plots the OWSBR's states over time and paints a more complete picture, showing how the OWSBR spends all control effort to tilt and yaw, without regard to stabilization.

PPO — Deterministic Episode Behaviour (2D view)

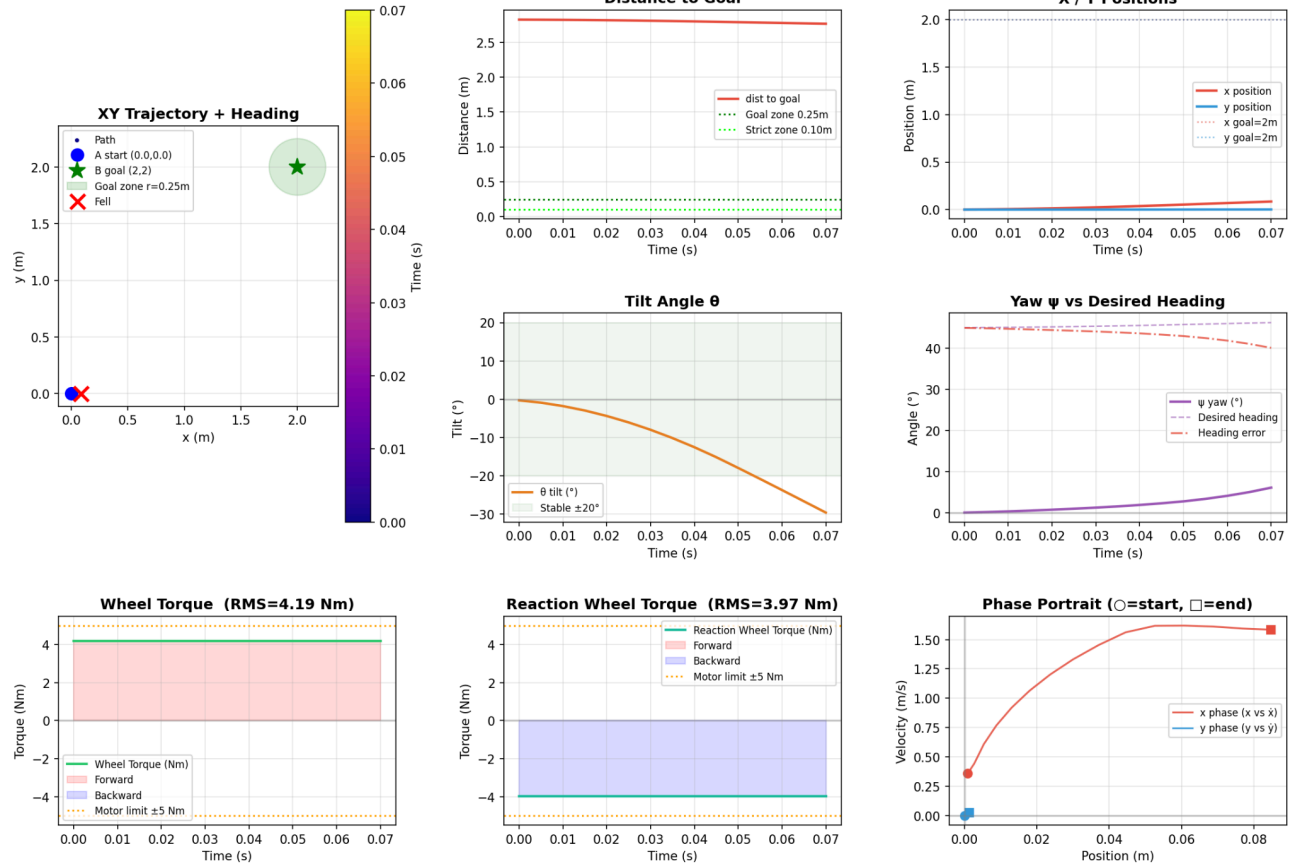


Fig 8. Model 2 with a reaction wheel states over time, trying to reach the goal.

Summary/Conclusion

The algorithms presented here performed well under the 1D traverse scenario, successfully reaching the target goal, albeit with performance differences. For Model 1, all algorithms performed well, with DDPG having less stability during training. For Model 2 (reaction wheel disabled), DDPG performed the best at reaching the goal, but at the cost of training stability and actuator oscillations compared to PPO and SAC. For Model 2 with the reaction wheel enabled, PPO performed poorly, even with extended model and training capacity. The additional degrees of freedom were too complex to capture, and the optimizer got stuck in a local maximum.

Previous work has divided a TWSBR controller into parts[18], balancing and translating separately, and provides inspiration for possible solutions. Given the success in this work with Model 1 and Model 2 (no reaction wheel) for traversing, it may be feasible to split a controller into parts:

- Yawing towards a point
- Traversing forwards along a 1D line

These could be trained separately, to later be combined for traversing a 2D plane.

Links to code

<https://github.com/OzzysMess/owsbr>

Contributions

Code & Analysis		
A.C	O.A.	E.C.
• PPO, SAC, and DDPG development and implementation	• Model 2 PPO	• Model 1 PPO, SAC, and DDPG implementation
Writing		
A.C	O.A.	E.C.
• Introduction • Methods • Results and Discussion	• Abstract • Conclusion • Methods • Results and Discussion	• Introduction • Methods • Results and Discussion • Bibliography

Bibliography

[1] M. A. Imtiaz, M. Naveed, N. Bibi, S. Aziz, and S. Z. H. Naqvi, “Control System Design, Analysis & Implementation of Two Wheeled Self Balancing Robot (TWSBR),” in 2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), 2018. doi: 10.1109/IEMCON.2018.8614858.

[2] G. Baskota, R. Devkota, S. Paneru, S. Yadav, D. Neupane, and O. P. Dhakal, “Analytical and Experimental Approach for Modeling, Simulation and Validation of Two-Wheeled Self-Balancing Robot,” in 2023 International Conference on Inventive Computation Technologies (ICICT), 2023. doi: 10.1109/ICICT57646.2023.10133983.

[3] M. Abdelrahim, M. A. Thabet, H. S. Abbas, M. M. M. Hassan, M. H. Amin, and A. Morsi, “Modeling and Control of a Ballbot: A Systematic Approach,” IEEE Access, vol. 13, pp. 141263–141280, 2025. doi: 10.1109/ACCESS.2025.3597797.

[4] S. Kordbacheh, K. Baghestan, and S. S. Gheidary, “Modeling and Robust Control of Ballbot Robot with Improved Power Transfer Mechanism,” in 2018 6th RSI International Conference on Robotics and Mechatronics (ICRoM), 2018, pp. 353–358. doi: 10.1109/ICRoM.2018.8657623.

- [5] Y. Rizal, T. Agustinah, R. Dikairono, and H. Du, "Design, Modeling, and Control of Unicycle Robots: A Systematic Literature Review," *IEEE Access*, vol. 14, pp. 20570–20588, 2026. doi: 10.1109/ACCESS.2026.3662281.
- [6] A. R. Geist et al., "The Wheelbot: A Jumping Reaction Wheel Unicycle," *arXiv preprint arXiv:2207.06988*, 2022. doi: 10.48550/arXiv.2207.06988.
- [7] Y. Li et al., "Adaptive Nonlinear Control Algorithm for a Self-Balancing Robot," *IEEE Access*, 2019. doi: 10.1109/ACCESS.2019.2963110.
- [8] N. G. M. Thao, D. H. Nghia, and N. H. Phuc, "A PID Backstepping Controller for Two-Wheeled Self-Balancing Robot," in *2010 International Forum on Strategic Technology (IFOST)*, 2010, pp. 76–81. doi: 10.1109/IFOST.2010.5668001.
- [9] H. Zhang and N. Mohamad Nor, "Fractional-Order PID Control of Two-Wheeled Self-Balancing Robots via Multi-Strategy Beluga Whale Optimization," *Fractal and Fractional*, vol. 9, no. 10, p. 619, 2025. doi: 10.3390/fractalfract9100619.
- [10] A. Xi, T. Wijekoon Mudiyanse, D. Tao, and C. Chen, "Balance Control of a Biped Robot on a Rotating Platform Based on Efficient Reinforcement Learning," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 4, pp. 938–951, 2019. doi: 10.1109/JAS.2019.1911567.
- [11] S. Levine, "Reinforcement Learning and Control as Probabilistic Inference: Tutorial and Review," *arXiv preprint arXiv:1805.00909*, 2018. doi: 10.48550/arXiv.1805.00909.
- [12] E. Masero, G. Mussita, A. La Bella, and R. Scattolini, "A Novel Reinforcement Learning-Based Approach for Optimal Control: An Application to Multi-Tank Water Systems," *Engineering Applications of Artificial Intelligence*, vol. 162, p. 112407, 2025. doi: 10.1016/j.engappai.2025.112407.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347.
- [14] Y. Kim, H. Jeon, and B.-Y. Kang, "Interactive Robot-Robot Reinforcement Learning for Object Balancing Task," in *2024 IEEE International Conference on Consumer Electronics (ICCE)*, 2024. doi: 10.1109/ICCE59016.2024.10444355.
- [15] T. Haarnoja et al., "Soft Actor-Critic Algorithms and Applications," *arXiv preprint arXiv:1812.05905*, 2018. doi: 10.48550/arXiv.1812.05905.
- [16] T. P. Lillicrap et al., "Continuous Control with Deep Reinforcement Learning," *arXiv preprint arXiv:1509.02971*, 2015. doi: 10.48550/arXiv.1509.02971.

[17] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 1587–1596, 2018. doi: 10.48550/arXiv.1802.09477

[18] R. Hernández, R. García-Hernández and F. Jurado, "Stabilization of a Two-Wheeled Self-Balancing Robot Using Reinforcement Learning," 2024 XXVI Robotics Mexican Congress (COMRob), Torreón, Coahuila, Mexico, 2024, pp. 117-122, doi: 10.1109/COMRob64055.2024.10777426